



3주차 과제

영역 · 자원	COW
고정하기	☐

3-1. 머신러닝 이론

다음 내용을 간략히 작성

- 머신러닝이란 무엇인가
데이터와 결과를 기반으로 스스로 패턴을 학습하고 이를 이용해서 예측하는 것
- 분류(Classification)와 회귀(Regression)의 차이
분류 모델은 레이블의 값들이 이산적으로 나뉘질 수 있는 문에 사용하는 것. 0 or 1 혹은 다중분류
회귀 모델은 레이블의 값들이 연속적인 문제에 사용. 연속적인 값 찾기
- 대표적인 머신러닝 모델 3가지와 특징
Linear regression 선형회귀 : 최적의 기울기를 구해서 가장 가까운 값 찾는 방법
Logistic regression 이진분류 : 0,1의 두 클래스를 분류하는 것으로 로지스틱 함수를 사용하여 이진 분류
Decision tree 의사결정 나무 : 데이터를 여러 조건에 따라 반복적으로 나누어 결과를 예측하는 모델, 트리구조여서 모델의 구조 파악이 쉬움. 분류와 회귀 작업이 가능.

<https://velog.io/@rhdmsco1163/이론실습편-Python을-활용한-AI-모델링-5-머신러닝-모델링>

3-2. 데이터 분할

다음 내용을 작성

- 학습 데이터(Train)
학습할때 필요한 데이터로 주요 컬럼과 타겟변수가 포함되어 있는 데이터

- 평가 데이터(Test)
모델의 성능을 파악하기 위해 사용하는 데이터로, 학습에 사용하지 않은 주요 컬럼과 타겟변수가 포함되어 있는 데이터
- 왜 데이터를 분리하여 학습하는지 설명
데이터를 분리하지 않으면 과적합 문제가 발생할 수 있으며, 학습에 사용하지 않은 데이터를 통해 모델의 실제 성능을 평가하기 위해 데이터를 분리함

3-3. 모델 학습

앞서 소개한 머신러닝 모델 중 1개 이상을 선택하여 아래 내용을 작성

- 사용한 하이퍼파라미터
 - random_state : 코드 실행 시 매번 동일한 결과를 얻기 위한 난수 생성 시드 (seed) 고정
 - n_estimators : 모델 학습에 사용할 전체 의사결정 나무(Decision Tree)의 개수 지정
 - max_depth : 트리의 최대 깊이를 제한하여 모델이 과도하게 학습되는 과적합 (Overfitting) 방지
 - min_samples_split : 노드를 다음 단계로 분할하기 위해 필요한 최소 샘플 개수 지정
 - min_samples_leaf : 리프 노드가 되기 위해 최소한으로 가져야 하는 샘플 개수 지정
 - class_weight : 클래스 불균형 문제를 가중치를 주어서 해결할 수 있음.
- 학습 과정

1. 모델 선정 및 데이터 준비

- Random Forest 분류 모델과 Logistic Regression 모델을 비교하여 최적의 모델을 선정하고자 함.
- 2주차에 생성한 train.csv와 test.csv에서 타겟 변수(is_profitable)를 제외한 피쳐들을 X_train, X_test로 설정하여 두 모델 학습 및 성능 평가 진행.

2. 클래스 불균형 문제 및 개선

- 현상 분석: 초기에 두 모델 모두 정확도는 높으나 재현율(Recall)이 낮게 나오는 현상 확인.

- **원인 파악:** 비행행 데이터(2,598개)가 흥행 데이터(1,279개)보다 약 2배 이상 많은 클래스 불균형 문제로 확인됨.
- **해결 방법:** `class_weight='balanced'` 하이퍼파라미터를 적용하여 데이터 비율에 맞춰 흥행 데이터에 더 높은 가중치를 부여해 학습시킴.

3. 최종 모델 선택 이유

- **모델 성능 비교:** Random Forest 모델은 Logistic Regression 모델에 비해 재현율이 약 0.08, F1-score가 약 0.009 다소 낮게 나타남.
- **최종 선택:** F1-score를 핵심 지표로 보았으나, 차이가 미미한 수준(0.009)인 반면 **Random Forest가 정확도와 정밀도가 높고 전체적인 모델의 변별 능력을 뜻하는 ROC-AUC가 더 우수함.**
- 따라서 잘못된 흥행 예측(False Positive) 리스크를 낮추고 전체적인 분류 안정성이 높은 **Random Forest 분류 모델을 최종 모델로 선택함.**

`class_weight='balanced'` 하이퍼 파라미터 부여 전 성능

```
=== [Random Forest Baseline 성능 평가] ===
- Accuracy (정확도) : 0.7856
- Precision(정밀도) : 0.7393
- Recall (재현율) : 0.5406
- F1-Score (F1점수) : 0.6245
- ROC-AUC : 0.8526

=== Confusion Matrix ===
[[589  61]
 [147 173]]
```

```
=== [Logistic Regression 성능 평가] ===
- Accuracy (정확도) : 0.7876
- Precision (정밀도) : 0.7143
- Recall (재현율) : 0.5938
- F1-Score (F1점수) : 0.6485
- ROC-AUC : 0.8371

=== Confusion Matrix ===
[[574  76]
 [130 190]]
```

`class_weight='balanced'` 하이퍼 파라미터 부여 후 성능

```
=== [Random Forest Baseline 성능 평가] ===
- Accuracy (정확도) : 0.7804
- Precision(정밀도) : 0.6636
- Recall (재현율) : 0.6781
- F1-Score (F1점수) : 0.6708
- ROC-AUC : 0.8561

=== Confusion Matrix ===
[[540 110]
 [103 217]]
```

```
=== [Logistic Regression 성능 평가] ===
- Accuracy (정확도) : 0.7670
- Precision (정밀도) : 0.6218
- Recall (재현율) : 0.7500
- F1-Score (F1점수) : 0.6799
- ROC-AUC : 0.8379

=== Confusion Matrix ===
[[504 146]
 [ 80 240]]
```

3-4. 모델 평가

다음 평가 지표에 대해 설명하고 결과를 작성

- Accuracy(정확도) : 가장 직관적으로 모델의 성능을 나타낼 수 있는 평가 지표

$$TP+TN / TP+TN+FP+FN$$
- Precision(정밀도) : 모델이 True라고 한 것중에서 실제 True인 비율

$$TP / TP+TN$$
- Recall(재현율) : 실제 True인것중에서 모델이 True라고 예측한 비율

$$TP/TP+FN$$
- F1-score(F1-점수) : 정밀도와 재현율의 조화 평균 (객관적 평가 가능)

$$2*(precision*recall)/(precision+recall)$$
- ROC : 모델의 분류 기준을 바꿔가면서 True Positive Rate(Recall, 참 양성률)과 False Positive Rate(거짓 양성률)의 관계를 나타낸 그래프
- AUC : ROC 곡선 아래의 면적을 의미하며, 모델이 두 클래스를 얼마나 잘 구분하는지 나타내는 지표
- Confusion Matrix(오차행렬) : 모델의 대략적인 성능 확인과 모델의 성능을 오차 행렬을 기반 수치로 표현
 - True Positive(TP) : True를 True로 예측(정답)
 - True Negative(TN) : False를 False로 예측(정답)
 - False Positive(FP) : False를 True로 예측(오답)
 - False Negative(FN) : True를 False로 예측(오답)

<https://m.blog.naver.com/snowstormaw/223219753823>

추가로

- 어떤 지표를 가장 중요하게 보았는지

F1-score를 중점적으로 확인하였고 보조 지표로는 ROC-AUC를 확인하였습니다.

비행행 데이터가 흥행 데이터에 비해서 2배 이상 많아서 정확도는 잘 나오지만 재현율이 잘 나오지 않아서 소수 클래스에 대한 정밀도와 재현율의 조화평균을 보아서 모델의 성능을 파악하였습니다.

- 모델이 어떤 경우에 오분류하는지 분석

피쳐 중요도 분석 결과, vote_count_log(30.2%), popularity_log(17.0%), vote_average(14.0%) 등 대중 반응 관련 변수 3개의 비중이 전체의 약 61.2%를 차지할 정도로 높게 나타났습니다.

이로 인해 모델은 투표 수와 인기도가 높으면 흥행(1), 낮으면 비흥행(0)으로 단순 판단하는 경향을 보입니다. 이에 따라 화제성은 높으나 손익분기점 달성에 실패한 대형작을 FP(False Positive)로 오판하거나, 투표 수는 적지만 저예산으로 높은 수익률을 기록한 영화를 FN(False Negative)으로 놓치는 구조적 오분류 한계가 발생함을 확인했습니다.